AUTOMATED REVIEW RATING SYSTEM

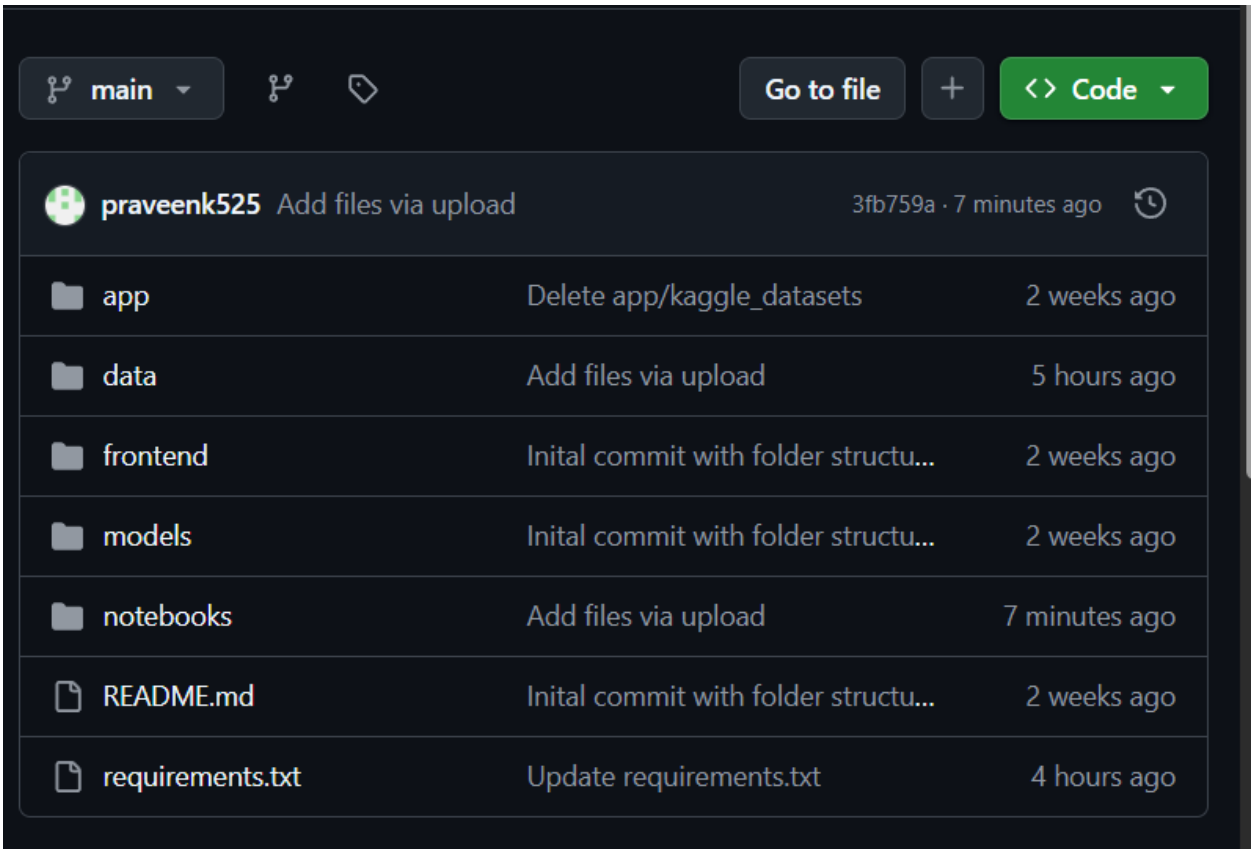
PROJECT OVERVIEW

This project automates the process of generating ratings from textual reviews. an AI-based system that automatically predicts the numerical rating (e.g., 1 to 5 stars) of a product, service, or business based solely on user-generated textual reviews. This aims to reduce rating manipulation, standardize feedback, and assist platforms in quickly gauging sentiment at scale.

ENVIRONMENT SETUP

- Installed python with necessary libraries such as numpy, pandas, matplotlib, scikit-learn, seaborn, nltk
- Colab

GITHUB PROJECT REPOSITORY



DATA COLLECTION

Data was collected from Kaggle and google datasets.

The project uses 10 diverse datasets across industries to ensure model generalization:

- Women's Clothing Reviews
- TripAdvisor Hotel Reviews
- Laptop Product Reviews
- IMDB Movie Reviews
- Amazon Product Reviews (including various categories)
- Plus 5 other domain-specific review datasets (e.g., food, electronics, services, etc.)

Each dataset contains:

- Review text
- Corresponding user rating (1–5 stars)
- Optional fields: title, product ID, date, reviewer info

All datasets are merged into one

Merged_dataset link=https://github.com/praveenk525/automated-review-rating-system/blob/main/data/merged_datasets/Merged_dataset.csv

DATA PREPROCESSING

Preprocessing is the first and most important step in any machine learning or data science project. It involves cleaning, transforming, and organizing raw data into a format that your machine learning model can understand and learn from effectively.

Load and Inspect Datasets

- Load all 10 datasets (e.g., Women's Clothing, TripAdvisor, etc.)
- Inspect structure: check column names, data types, sample records
- Verify presence of review and rating columns

`pd.read_csv()` → Load CSV files into a DataFrame

`pd.read_excel()` → For Excel files

```
# View first few rows

df1.head()

# View last few rows

df1.tail()

# Check column names

df1.columns

# Get DataFrame shape (rows, columns)

df1.shape

# Check column data types

df1.dtypes

# Full summary info

df1.info()

# Summary statistics (for numeric columns)

df1.describe()
```

Remove Unnecessary Columns

- Drop columns not required for prediction, such as:
    - user_id, product_id, timestamp, title, image_url, etc.
- Keep only relevant columns: usually review_text and rating

Remove Null or Missing Values

- Remove rows with null (missing) values in review_text or rating

```
 # Check for missing values in each column

print(df.isnull().sum())

# removing null values
```

df.dropna(inplace=True)

Remove rows where review_text or rating is missing

df.dropna(subset=['review_text', 'rating'], inplace=True)

Remove Duplicates and Conflicting Reviews

- **Exact duplicates**: same review and rating — keep one
- **Conflicting reviews**: same review text, different ratings — remove all

#check duplicates

print(df.duplicated().sum())

Remove all duplicate rows

df.drop_duplicates(inplace=True)

Clean Text Data

- Convert text to lowercase
- Remove:
    - URLs
    - Punctuation
    - Numbers and special characters
    - Extra spaces

```
import re

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"[^\w\s]", "", text)
    text = re.sub(r"\d+", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df.loc[:, "review"] = df["review"].apply(clean_text)
```

Stopwords Removal

Stopwords are common words in a language (like *"the"*, *"is"*, *"at"*, *"which"*, *"and"*) that do not add significant meaning to text and are usually removed during text preprocessing in NLP tasks.

Why Remove Stopwords?

- They don't help in understanding sentiment or meaning.
- They inflate feature space and slow down training.
- Removing them improves model accuracy and efficiency.

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
nltk.download('stopwords')
nltk.download('punkt')

stop_words = set(stopwords.words('english'))

def remove_stopwords(text):
    words = word_tokenize(str(text))  # tokenize text
    filtered = [word for word in words if word.lower() not in stop_words]
    return ' '.join(filtered)


df['review']=df['review'].apply(remove_stopwords)
```

What is Lemmatization?

Lemmatization is the process of reducing a word to its root form (lemma) based on its dictionary meaning and part of speech.

Example:

- "running" → "run"
- "better" → "good"
- "was" → "be"

Lemmatization uses a vocabulary (dictionary) and morphological analysis to correctly identify the base form of a word.

What is Stemming?

Stemming is a simpler, rule-based process that chops off prefixes or suffixes to get to the root word. It doesn't care about grammar or word meaning.

Example:

- "running" → "run"
- "flying" → "fli"
- "studies" → "studi"

Why Lemmatization is Better (in most ML tasks):

- Returns actual words (important for readability and clarity).
- More grammatically correct results.
- Handles irregular words better (e.g., "better" → "good").
- Reduces noise compared to stemming.

```
nltk.download('wordnet')
nltk.download('omw-1.4')
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
lemmatizer = WordNetLemmatizer()
def lemmatize_text(text):
    words = word_tokenize(text)
    lemmatized = [lemmatizer.lemmatize(word) for word in words]
    return ' '.join(lemmatized)
df['review']=df['review'].apply(lemmatize_text)
```

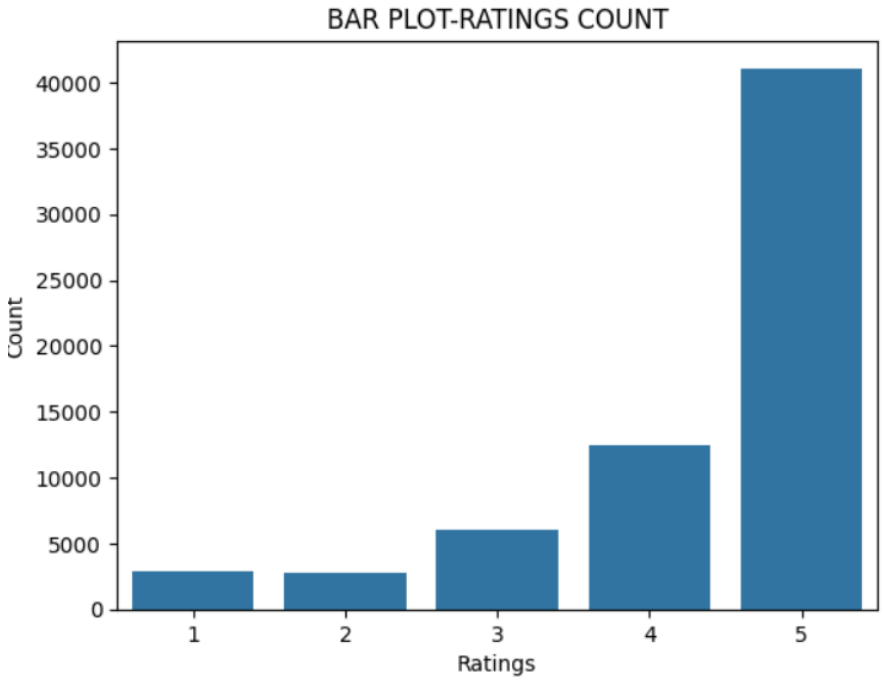
DATA VISUALIZATION

Data Visualization is the process of representing data and information visually using charts, graphs, and other visual elements (like maps or word clouds) to help people understand patterns, trends, and insights in data.

BAR CHART

A bar chart represents categorical data with rectangular bars. The length of each bar is proportional to the value it represents.

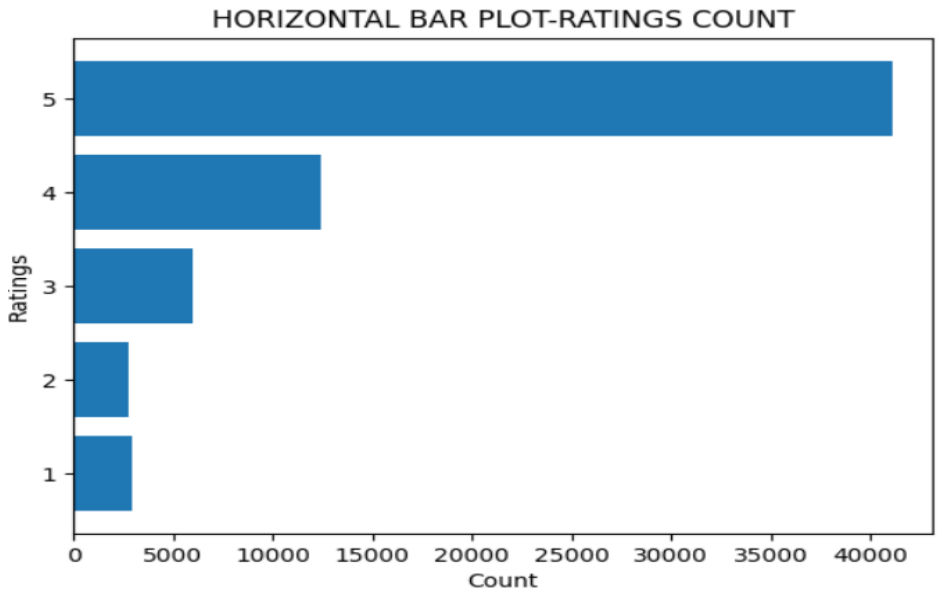
- Use: To show the number of reviews in each star rating (e.g., how many 1-star, 2-star, etc.).
- Example: Number of reviews per rating (1 to 5).



Horizontal Bar Chart (HBar)

A horizontal bar chart is like a bar chart, but the bars extend horizontally instead of vertically.

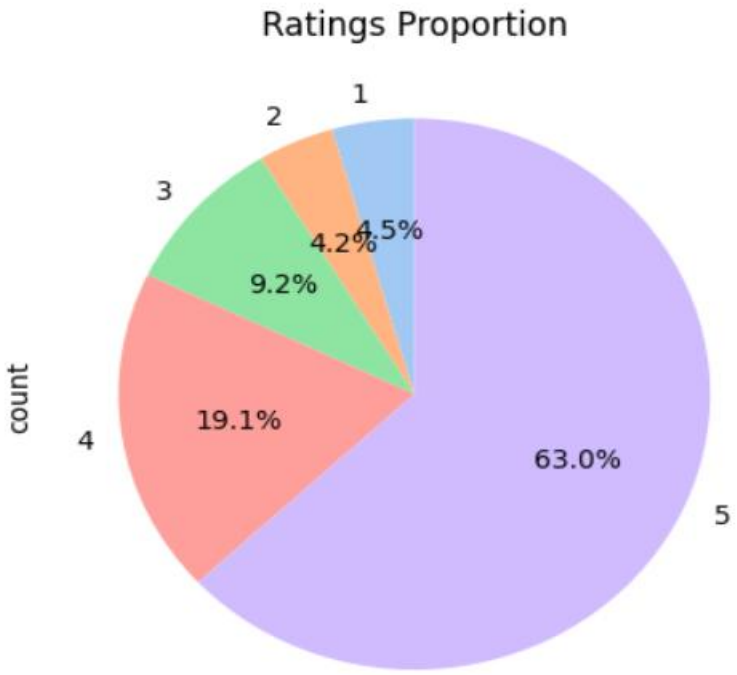
- To show average values like review length for each rating in a clean, readable format.
- Example: Average number of words in 5-star reviews vs 1-star.



Pie Chart

A pie chart is a circular chart divided into slices to illustrate proportions of categories.

- To show the percentage of each rating (how much of the data is 5-star, 4-star, etc.).
- Example: 50% of reviews are 5-star, 10% are 1-star.

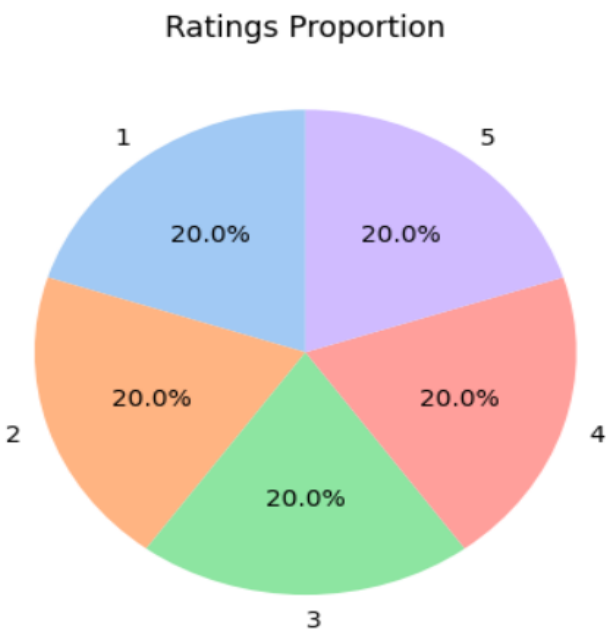


BALANCED DATASET

A balanced dataset is one in which all classes or categories have an equal (or nearly equal) number of samples. This ensures that the machine learning model learns from all classes equally and doesn't become biased.

Balanced dataset link=https://github.com/praveenk525/automated-review-rating-system/blob/main/data/balance_imbalance_datsets/Balanced_dataset.csv

Balanced dataset with 2000 reviews and ratings



IMBALANCED DATASET

An imbalanced dataset is one in which some classes have significantly more samples than others.This can lead to a biased model, which performs well on the majority class but poorly on the minority class.

Imbalanced dataset link=https://github.com/praveenk525/automated-review-rating-system/blob/main/data/balance_imbalance_datsets/imbalanced-data.zip

Imbalanced dataset contains